SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA51
COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Analytical Problem Solving: 40%

QUESTIONS: **Total Marks: 50**

Annexure A

Code of Conduct:

*I **DHARANIDARAN D (192521337)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Dharanidaran D
Signature of the student:

Annexure A Analytical Problem Solving

1.RSA Encryption Analysis

Given p=11, q=13

a. Compute n ϕ(n)

b. Choose e=7 and find the private key d

c. Encrypt the message M=9

d. Decrypt the ciphertext and verify the result

e. Analyze how changing e affects security

Cryptographic Parameter Estimation and Verification

RSA Public and Private Key Generation

Computation of Modulus and Totient

The mathematical basis of RSA requires calculating the system modulus n and Euler's totient function $\phi(n)$ using the given prime numbers $p = 11$ and $q = 13$:

- Modulus: $n = p \times q = 11 \times 13 = 143$
- Totient: $\phi(n) = (p - 1) \times (q - 1) = (11 - 1) \times (13 - 1) = 10 \times 12 = 120$

Derivation of the Private Key

The private key exponent d is the multiplicative inverse of the public exponent e modulo $\phi(n)$. Given $e = 7$:
$d\times e\equiv 1\mathinner{\;\left(\mod \,\phi (n)\right)}\implies 7d\equiv 1\mathinner{\;\left(\mod \,120\right)}$
Using the Extended Euclidean Algorithm:
$120=7\times 17+1\implies 1=120-7\times 17$
$7\times (-17)\equiv 1\mathinner{\;\left(\mod \,120\right)}$
$d\equiv -17+120=103$
The calculated private key exponent is $d = 103$.

Encryption and Decryption Operations

Message Encryption Process

The plaintext message $M = 9$ is transformed into ciphertext C using the public key pair $(e, n) = (7, 143)$ via modular exponentiation:

$C=M^{e}\mathinner{\;\left(\mod \,n\right)}=9^{7}\mathinner{\;\left(\mod \,143\right)}$

- $9^1 \equiv 9 \pmod{143}$
- $9^2 \equiv 81 \pmod{143}$
- $9^3 = 729 = (5 \times 143) + 14 \equiv 14 \pmod{143}$
- $9^6 = (9^3)^2 \equiv 14^2 = 196 \equiv 53 \pmod{143}$
- $9^7 = 9^6 \times 9^1 \equiv 53 \times 9 = 477 = (3 \times 143) + 48 \equiv 48 \pmod{143}$

The resulting ciphertext is $C = 48$.

Ciphertext Decryption and Verification

The ciphertext $C = 48$ is decrypted back into the plaintext message $M^{\prime }$ using the private key pair $(d, n) = (103, 143)$:

$M^{\prime }=C^{d}\mathinner{\;\left(\mod \,n\right)}=48^{103}\mathinner{\;\left(\mod \,143\right)}$

Applying the square-and-multiply strategy based on the binary representation of $103 = 64 + 32 + 4 + 2 + 1$:

- $48^1 \equiv 48 \pmod{143}$
- $48^2 = 2304 \equiv 16 \pmod{143}$
- $48^4 \equiv 16^2 = 256 \equiv 113 \pmod{143}$
- $48^8 \equiv 113^2 = 12769 \equiv 42 \pmod{143}$
- $48^{16} \equiv 42^2 = 1764 \equiv 48 \pmod{143}$ (Cycle begins)
- $48^{32} \equiv 16 \pmod{143}$
- $48^{64} \equiv 113 \pmod{143}$

$M^{\prime }=48^{64}\times 48^{32}\times 48^{4}\times 48^{2}\times 48^{1}\equiv 113\times 16\times 113\times 16\times 48\mathinner{\;\left(\mod \,143\right)}$

$M^{\prime }\equiv (113\times 16)\times (113\times 16)\times 48\equiv 92\times 92\times 48\equiv 27\times 48=1296\equiv 9\mathinner{\;\left(\mod \,143\right)}$

Since $M' = 9$, the decrypted value perfectly matches the original plaintext message $M = 9$.

Security Impact Analysis

Structural Exponent Impact

The choice of the public exponent e dictates both operational performance and resistance to targeted mathematical attacks:

- Small Exponent Risks: Selecting a very small value like $e = 3$ speeds up verification but creates exposure to Coppersmith's attack or Broadcast attacks if the same message is transmitted across multiple networks without sufficient padding.

- Standard Selection Benefits: Utilizing a standard value such as \(e = 65537\) (\(2^{16}+1\)) guarantees excellent security against low-exponent structural attacks while preserving high computational efficiency due to its low Hamming weight.

2.Block Cipher Mode Evaluation

Given plaintext blocks:

P=[1010,1010,1111,1010], Key = K=1100

a. Encrypt using ECB and CBC modes (Assume IV = 0000)

b. Compare the ciphertext outputs

c. Analyze which mode hides patterns better and explain why

Comparative Analysis of Block Cipher Operating Modes

Bitwise Mode Encryption Mechanics

Electronic Codebook (ECB) Mode Encryption

ECB mode processes each plaintext block independently using the core cipher function, implemented here as a standard bitwise exclusive-OR (XOR) operation (\(C_i = P_i \oplus K\)):

- Block 1: \(C_1 = 1010 \oplus 1100 = 0110\)
- Block 2: \(C_2 = 1010 \oplus 1100 = 0110\)
- Block 3: \(C_3 = 1111 \oplus 1100 = 0011\)
- Block 4: \(C_4 = 1010 \oplus 1100 = 0110\)

The complete ECB ciphertext sequence is \(C_{ECB} = [0110, 0110, 0011, 0110]\).

Cipher Block Chaining (CBC) Mode Encryption

CBC mode introduces dependencies by XORing each plaintext block with the preceding ciphertext block prior to encryption (\(C_i = (P_i \oplus C_{i-1}) \oplus K\)), initialized with \(C_0 = IV = 0000\):

- Block 1: \(C_1 = (1010 \oplus 0000) \oplus 1100 = 1010 \oplus 1100 = 0110\)
- Block 2: \(C_2 = (1010 \oplus 0110) \oplus 1100 = 1100 \oplus 1100 = 0000\)
- Block 3: \(C_3 = (1111 \oplus 0000) \oplus 1100 = 1111 \oplus 1100 = 0011\)

- Block 4: $C_4 = (1010 \oplus 0011) \oplus 1100 = 1001 \oplus 1100 = 0101$
  The complete CBC ciphertext sequence is $C_{CBC} = [0110, 0000, 0011, 0101]$.

  Behavioral Output Comparison

  Structural Mapping Contrasts

  The structural characteristics of the generated output vectors vary significantly between the two configurations:
- ECB Observation: The repeating plaintext patterns ($P_1 = P_2 = P_4 = 1010$) produce identical ciphertext blocks ($C_1 = C_2 = C_4 = 0110$).
- CBC Observation: Despite identical source blocks, the resulting ciphertext segments are entirely distinct ($C_1 = 0110, C_2 = 0000, C_4 = 0101$) due to the dynamic feedback loop.

Pattern Diffusion Evaluation

Vulnerability and Diffusion Rationale

CBC mode delivers drastically superior data obfuscation capabilities compared to ECB mode:

- ECB Weakness: Because it lacks inter-block diffusion, ECB directly mirrors structural redundancies of the plaintext into the ciphertext, allowing eavesdroppers to deduce underlying data patterns without breaking the key.
- CBC Strength: The integration of an Initialization Vector and previous block feedback ensures that the context of each block shifts dynamically. This mechanism entirely conceals repeating patterns, providing semantic security essential for real-world cryptographic applications.

3.DES vs 3-DES vs Blowfish Performance Study

Given:

a. DES key size = 56 bits, time = 2 ms/block

b. 3-DES key size = 168 bits, time = 6 ms/block

c. Blowfish key size = 128 bits, time = 1 ms/block

For 1000 blocks of data:

i.Calculate total encryption time for each algorithm
ii.Analyze trade-offs between security and performance
iii.Recommend the best algorithm for secure real-time communication

Performance Evaluation and Algorithmic Benchmarking

Execution Time Calculations

Cumulative Computational Latency

The total encryption runtime required to process a workload of $N = 1000$ data blocks is computed for each candidate algorithm below:

- Data Encryption Standard (DES): $T_{DES} = 1000 \text{ blocks} \times 2 \text{ ms/block} = 2000 \text{ ms} = 2.0 \text{ seconds}$
- Triple DES (3-DES): $T_{3DES} = 1000 \text{ blocks} \times 6 \text{ ms/block} = 6000 \text{ ms} = 6.0 \text{ seconds}$
- Blowfish: $T_{Blowfish} = 1000 \text{ blocks} \times 1 \text{ ms/block} = 1000 \text{ ms} = 1.0 \text{ second}$

Security and Performance Trade-offs

Architectural Balancing Analysis

The comparative trade-offs highlight the distinct operational focus of each system architecture:

- DES Profile: Offers moderate throughput but suffers from an unacceptably small 56-bit key space that is highly vulnerable to modern parallelized brute-force attacks.
- 3-DES Profile: Enhances cryptographic strength by extending the key size to 168 bits, but incurs a heavy computational penalty, executing the internal pipeline three times per block and creating latency bottlenecks.
- Blowfish Profile: Achieves an optimal design balance, providing a highly secure, variable key length (up to 448 bits, evaluated at 128 bits) alongside an ultra-fast software implementation that minimizes CPU overhead.

Engineering Recommendation

Optimal Real-Time Selector

Blowfish is explicitly recommended for real-time communication systems:

- Rationale: Real-time environments demand minimized latency and deterministic execution pathways. Blowfish successfully meets these requirements by delivering the lowest total processing time (1.0 second), while its robust 128-bit key architecture guarantees complete data confidentiality against modern cryptographic attacks.

4.Diffie-Hellman Key Exchange Analysis

Given:

a. p=23, g=5

b. User A private key = 6

c. User B private key = 15

d. Compute public keys for A and B

e. Compute the shared secret key

f. Demonstrate how a Man-in-the-Middle attacker could alter the exchange

g. Suggest a secure method to prevent the attack

Cryptographic Key Exchange and Threat Modeling

Key Derivation Mathematical Calculations

Verification of User Public Keys

Public keys are derived by raising the generator $g = 5$ to the power of the respective private key modulo $p = 23$:

- User A Public Key (Y_{A}): $Y_A = g^{X_A} \pmod p = 5^6 \pmod{23}$
  $5^{2}=25\equiv 2\mathinner{\;\left(\mod \,23\right)}\implies 5^{4}\equiv 2^{2}=4\mathinner{\;\left(\mod \,23\right)}$
  $Y_{A}=5^{4}\times 5^{2}\equiv 4\times 2=8\mathinner{\;\left(\mod \,23\right)}$
- User B Public Key (Y_{B}): $Y_B = g^{X_B} \pmod p = 5^{15} \pmod{23}$
  $5^{6}\equiv 8\mathinner{\;\left(\mod \,23\right)}\implies 5^{12}\equiv 8^{2}=64\equiv 18\mathinner{\;\left(\mod \,23\right)}$
  $Y_{B}=5^{12}\times 5^{2}\times 5^{1}\equiv 18\times 2\times 5=180\mathinner{\;\left(\mod \,23\right)}$
  $180=(7\times 23)+19\implies Y_{B}=19$

Computation of the Shared Secret Key

The shared secret key K is computed by raising the counterpart's public key to the user's private key modulo p:

- Calculation by User A: $K = Y_B^{X_A} \pmod p = 19^6 \pmod{23}$

$19\equiv -4\mathinner{\;\left(\mod \,23\right)}\implies 19^{2}\equiv (-4)^{2}=16\equiv -7\mathinner{\;\left(\mod \,23\right)}$
$19^{4}\equiv (-7)^{2}=49\equiv 3\mathinner{\;\left(\mod \,23\right)}\implies 19^{6}=19^{4}\times 19^{2}\equiv 3\times 16=48\mathinner{\;\left(\mod \,23\right)}$
$48=(2\times 23)+2\implies K=2$

- Calculation by User B: $K = Y_A^{X_B} \pmod p = 8^{15} \pmod{23}$
$8^{2}=64\equiv 18\equiv -5\mathinner{\;\left(\mod \,23\right)}\implies 8^{3}=8^{2}\times 8\equiv -5\times 8=-40\equiv 6\mathinner{\;\left(\mod \,23\right)}$
$8^{6}\equiv 6^{2}=36\equiv 13\mathinner{\;\left(\mod \,23\right)}\implies 8^{12}\equiv 13^{2}=169\equiv 8\mathinner{\;\left(\mod \,23\right)}$
$K=8^{12}\times 8^{3}\equiv 8\times 6=48\equiv 2\mathinner{\;\left(\mod \,23\right)}$
Both entities successfully derive the identical shared secret key, $K = 2$.

Adversarial Threat Modeling

Man-in-the-Middle (MitM) Attack Mechanics

The classic Diffie-Hellman protocol lacks native device authentication, allowing an active attacker (Darth) to compromise the exchange link:

- Interception Phase: Darth intercepts User A's public value Y_{A} and substitutes it with an adversarial public key Y_{D1}. Similarly, Darth intercepts User B's public value Y_{B} and substitutes it with Y_{D2}.
- Compromise State: User A unknowingly establishes a shared key K_{AD} with the attacker, while User B establishes key K_{BD} with the attacker. Darth can now transparently decrypt, read, manipulate, and re-encrypt the data flowing between both victims without triggering system alerts.

Network Defense Remediation

Authentication Frameworks

To guarantee key exchange integrity, the communication channel must enforce explicit identity verification:

- Public Key Certificates: Integrating a trusted Public Key Infrastructure (PKI) allows both users to wrap their public keys in digital signatures, ensuring that anomalous public values injected by an attacker are rejected instantly.

- Station-to-Station (STS) Protocol: Utilizing the STS protocol combines classic Diffie-Hellman with cryptographic signatures and session-based validation tokens to effectively neutralize active interception vectors.

5.ECC vs RSA Comparison

Given:

a. RSA key size = 1024 bits

b. ECC key size = 160 bits (equivalent security)

c. Device processing capacity = limited (IoT device)

d. Analyze computational cost for both systems

e. Estimate which consumes less power and memory

f. Justify which cryptosystem is better for the given scenario with reasoning

Architectural Feasibility Analysis for Resource-Constrained Systems

Algorithmic Computational Complexity

Mathematical Processing Overhead

The primary structural divergence between RSA and Elliptic Curve Cryptography (ECC) centers on the underlying mathematical frameworks:

- RSA Constraints: Requires performing complex modular exponentiation on massive 1024-bit integers. As key sizes scale, the processing requirement increases cubically (\(O(k^3)\)), creating significant operational delays on lightweight processor architectures.
- ECC Advantages: Relies on elliptic curve point multiplication across much smaller 160-bit algebraic fields. These compact algebraic structures require far fewer processor instruction cycles, accelerating cryptographic operations.

Hardware Resource Consumption

Power and Memory Footprint Estimation

The physical footprint of the runtime environment is highly sensitive to the key dimensions chosen:

- Memory Allocation: ECC utilizes a compact 160-bit key size, which dramatically reduces the RAM buffer space and flash storage capacity required compared to the bulky 1024-bit parameters needed for RSA.

- Power Budget Consumption: Because RSA forces the CPU to run for prolonged periods during verification and decryption operations, it drains battery power rapidly. Conversely, ECC's rapid processing minimizes CPU uptime, significantly conserving power.

System Selection Justification

Definitive Cryptosystem Recommendation

ECC is the ideal choice for resource-constrained Internet of Things (IoT) deployments:

- Engineering Rationale: IoT nodes typically operate under strict energy and memory limitations. ECC provides equivalent cryptographic strength to an RSA 1024-bit system using a key that is nearly seven times smaller. This reduction minimizes network packet transmission overhead, preserves valuable memory space, and drastically cuts power consumption—maximizing the operational lifetime of the edge device deployment.
- ss.
- If you need the exact **rubric breakdown** mapped to specific point allocations for grading.
- If you require another key variant (such as **RSA-2048** or **ECC-256**) evaluated for compliance with updated security standards.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/form	Selects correct method	Inappropriate or partially correct	Unable to select method

		ula with strong justification	with minor errors	method	
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation